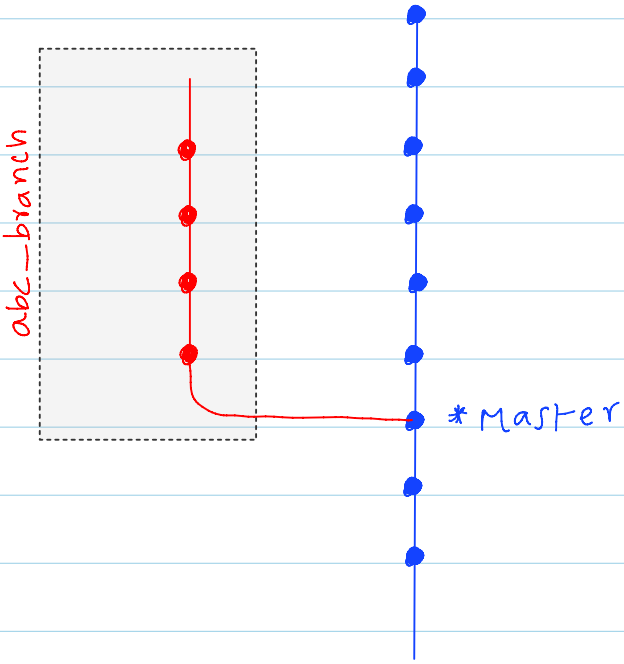
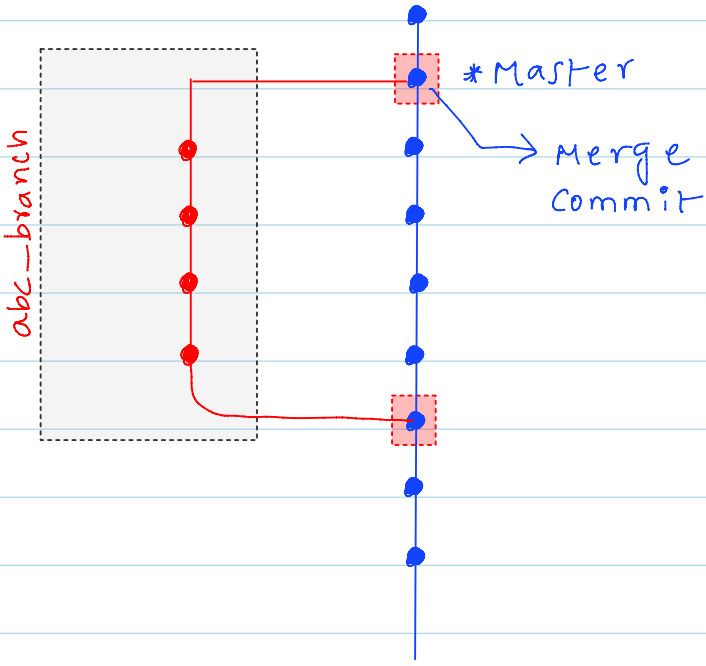
Branching in Git

→ Branch is nothing but a diversion from the Development (Concurrent Independent Development)

→ suppose we are working on a webApplication and we have to add a Navbar and a Card and we have 2 developers so that one developer will work on one functionality and other one can work up on other at same time

→ this is called branching





→ here we have created a seperate branch for developing New feature and

→ red branch is progressing and Blue branch also getting progress

→ after finishing all development we are merging the work done by both developers.

Commands and Details :-

→ to list all branches

>> git branch

→ so now we are in master (main) branch and we need to create a new branch

→ for that first we have to enter *master (main/) branch ← both are same

new Command after 2019 → >> git switch main ← Any branch name you want to enter

→ we can only switch between branches

old Command → >> git checkout main

☆ with git checkout we can do so many things creating branch, add/delete files

* Creating a New branch

→ we can use either of the commands

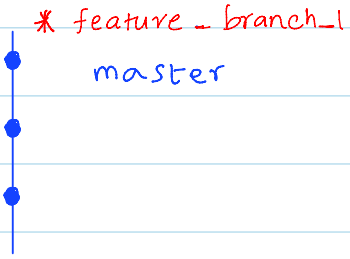
>> git checkout -b <branch-name>

→ if there is a branch exists with name it don't create, if not it will create a new one

>> git switch -c <branch-name>

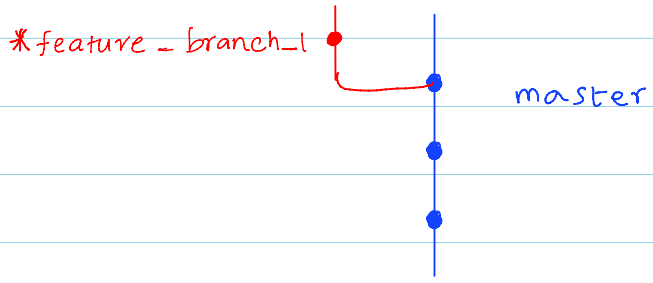
→ if there is a branch switched to that
→ if not creates a new one and switch

workflow



>> git switch -c feature_branch_1

→ now we are in feature_branch-1 and we are made some changes and commit that



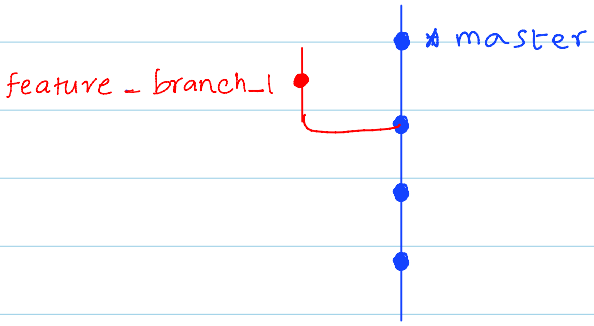
>> git add .
>> git commit -m "new data"

→ now if we switch to main branch and inspect all the files we can't be able to see the changes made in previous branch

→ so make another change in new file and commit

>> git switch main

→ Make some changes in other file



>> git add .

>> git commit -m "from main"

Create a Senario

→ we have a folder structure and we will be creating 3 branches and work up on independent files

- file1.txt
- file2.txt
- file3.txt
- main.txt

→ so we will create 3 branches
main, feature-branch-1
feature-branch-2

→ from main → file1.txt
→ from feature-branch-1 → file2.txt
→ from feature-branch-2 → file3.txt

→ from main add some data and commit

file1 ⇒

```
Hello this is main
adding data to file1
```

>> git checkout -b feature-branch-1

file2 ⇒

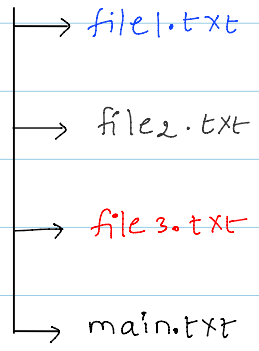
```
feature-branch-1 adding
data to file2
```

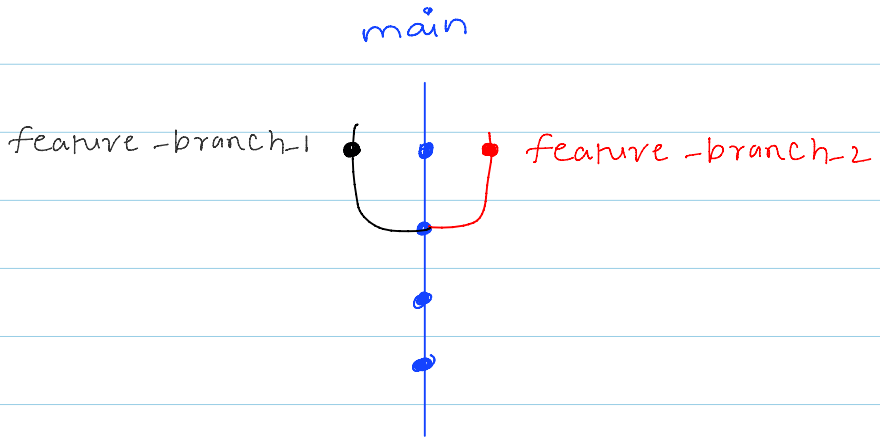
>> git checkout -b feature-branch-2

file3 ⇒

```
feature-branch-1 adding
data to file3
```

→ now we had three files in our folder and three branches look like this





→ now one Important point to consider is that

main don't know about [feature-branc-1 / feature-branch-2] changes

feature-branc-1 don't know about [main / feature-branch-2] changes

feature-branch-2 don't know about [main / feature-branc-1] changes

→ They are Independent to each other and changed in independent files (No common file)

→ So that is No problem while Merging.

→ now i wanted to merge all the changes that are done by other 2 branches to main

Steps:

① go to main branch

>> git switch main.

② merge two branches by writing a message

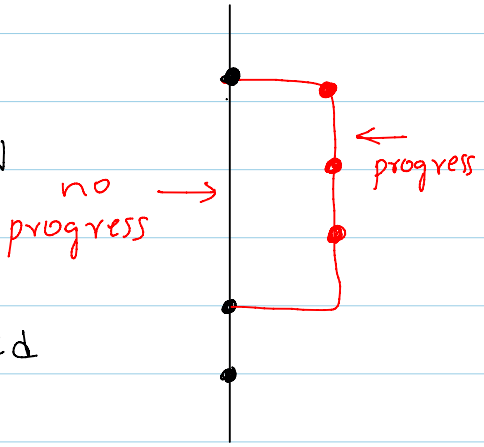
>> git merge <branch-name> -m "message"

Types of merges:-

1) fast forward merge

→ when we created a branch and that branch is progress but not main

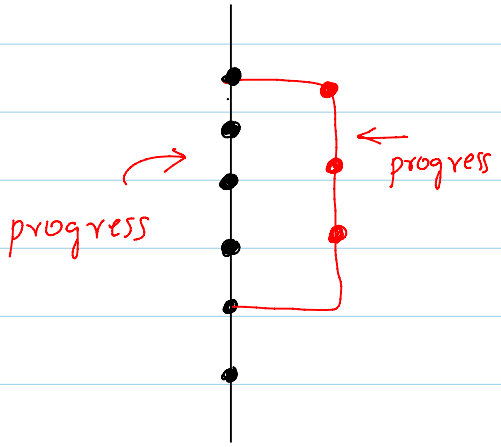
→ if we merge them it is called as fast forward merge



2) No fast-forward Merge:-

→ when the new branch and main branch getting progress at same time differently and we are Merging them

→ That is No-fast forward merge



3) Three way merge

→ we have divided into 2 branches main and feature

→ and